

StoreMind: What we changed, what we built, and what comes next

A simple guide for the whole team. Read it in about 10 minutes, then send your suggestions.

What is inside

1. The 1-page summary
2. How StoreMind works (the big picture)
3. Software: what we changed and why
4. Results so far (real numbers, told honestly)
5. Hardware: what each part does
6. Connecting to a shop's CCTV cameras
7. What makes us different (novelty)
8. Problems we expect, and how we will fix them
9. What comes next, decisions we need, and how you can help
10. Simple glossary of technical words

Where everything lives: GitHub repo `Vinith-44/SIH` (private). Start with `research/00_START_HERE.md`. Code is in `storemind/`. Results are in `storemind/storemind/eval/results/RESULTS.md`. Date of this update: 24 September 2026.

1. The 1-page summary

StoreMind in one line: a small box (Raspberry Pi 5 now, a Qualcomm board later) that connects to the CCTV a shop already has, and tells the owner **how many people came in, how long the billing queue is, and which shelf is empty**. It works **without internet**, and it **never saves video or faces**.

What we did

Area	What changed
Checked the old code	Found the big problems: the queue "prediction" gave false alarms (it predicted 78 people from a queue of 1), the shelf model memorised its training images instead of learning (overfitting), counting failed on our own test video, and the PPT had claims the code did not do.
Software	Rebuilt everything as one system instead of 3 separate scripts: camera input → AI detection → tracking → counting, queue, shelf → database → live dashboard → alerts. 124 automatic tests pass.
Real results	Tested on real mall CCTV (CAVIAR dataset). The old code counted 95 people entering when the truth was 30 . The new code counts 33 . Entry accuracy 90% , exit 85.7% (exits still need work).
Hardware plan	Pi 5 = the brain (AI). STM32 = the nerves (weight sensors, distance sensor, door beam, radar, tower light). Plan to move the AI to a Qualcomm board, because the sponsor is Qualcomm.
Research	More than 100 sources: Qualcomm's plans, competitors, NVIDIA/Intel designs, Indian CCTV market, datasets, problems we will face. All in the <code>research/</code> folder.
GitHub	One private repo for everything. Work on your own branch, then send a pull request.

What we are doing next

- Fix the exit over-count, make queue tracking smarter, add the STM32 sensor link and the ₹999 radar.
- Connect to real shop CCTV (and a cheap Wi-Fi camera for the demo).
- Run the AI on a Qualcomm chip and measure speed and power.
- Record our own videos (canteen queue, shelf) to get real accuracy for queue and shelf.

What we need from you

Read

this document, sections 5–9 at least

Answer

the 5 decisions in section 9

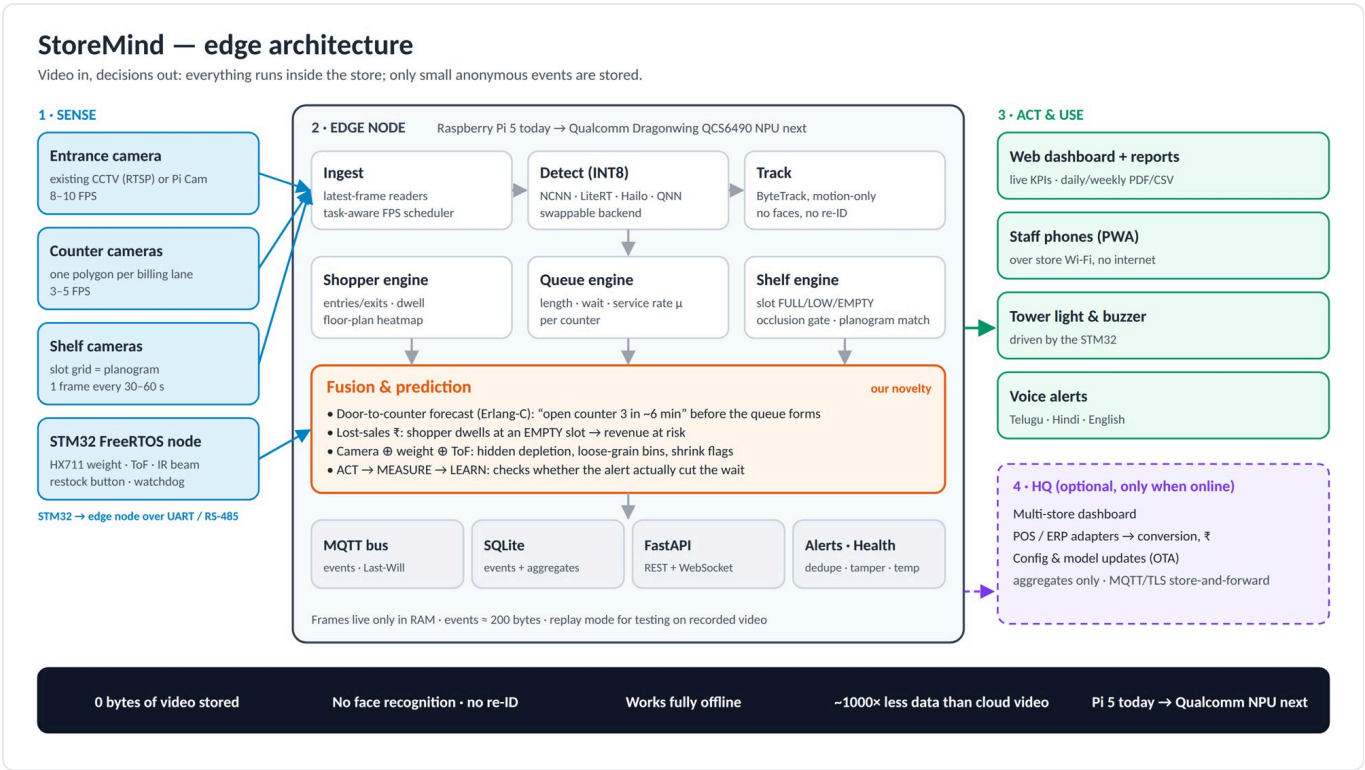
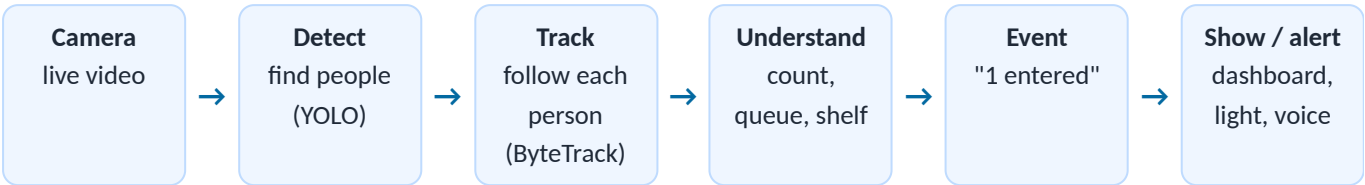
Suggest

changes via GitHub pull request or our group chat

2. How StoreMind works (the big picture)

Think of the shop as a body:

Body part	In StoreMind	What it does
Eyes	Cameras (the shop's CCTV or a Pi camera)	See the door, the billing counter and the shelves
Brain	Raspberry Pi 5 → Qualcomm board	Runs the AI, turns video into small "events" like "1 person entered"
Nerves	STM32 microcontroller + sensors	Feels shelf weight, a hand near the shelf, the door beam, people at the counter (radar). Reacts instantly
Memory	Small database (SQLite)	Stores only events, never video
Mouth	Dashboard, phone alerts, tower light, voice	Tells staff "open counter 2" or "refill shelf A3"



Full architecture. Left: sensors and cameras. Middle: the edge box (all AI runs here, frames stay in memory). Right: dashboard, alerts, and head office (optional, only summaries go up).

Privacy rule for everyone: no face recognition, no saving video, no identity. We only count and time anonymous people. This is also our pitch to the judges.

3. Software: what we changed and why

Old code vs new code

Old problem	What we did	Status
3 separate laptop scripts, Windows only	One system that runs on laptop and on the Pi (no screen needed)	done
Frames squashed into a 320×320 square (people looked stretched)	Correct resize (letterbox), better model (YOLO11n / YOLO26n)	done
Weak home-made tracker, lost people	ByteTrack tracker (standard, fast)	done
Counted using the middle of the body box	Counts using the feet point , with a buffer zone around the line	done
Queue alarm from 1 person ("78 predicted")	Warm-up time, smoothing, proper wait-time and service-time timers	done
No queue prediction	Forecast from the entrance using a queueing formula (Erlang-C)	done
Shelf model overfitted, no accuracy numbers	New method: compare each shelf slot with a "full shelf" photo; ignore frames where a person blocks the shelf	v1 done
Logs deleted every run, no database	SQLite database, daily and hourly totals	done
Fake "DEMO" dashboard in PPT	Real web dashboard (works offline, works on phones)	done
No way to prove accuracy	Evaluation system: every number comes from a command we can re-run	done
Zones were guesses	Calibration tool: click on a camera photo to draw lines and zones	done
No STM32 connection in code	Serial bridge + simulator + FreeRTOS firmware	next
Never tested on Qualcomm	Qualcomm AI Hub script + run on Qualcomm board	next

Code map (folder → job)

Folder in storemind/storemind/	Job, in simple words
ingest/	Reads cameras (USB, Pi camera, CCTV over RTSP, video files). Always keeps only the newest frame so nothing lags.
inference/	The AI detector. Can switch between engines (Ultralytics, LiteRT/TFLite, ONNX) without changing other code.
tracking/	Follows each person between frames (ByteTrack).
analytics/	<code>football.py</code> (in/out counting), <code>zones.py</code> (dwell time), <code>heatmap.py</code> (where people walk), <code>queue.py</code> (queue length, wait, service), <code>shelf.py</code> (shelf slot full/low/empty).
fusion/	<code>forecast.py</code> (predict queue from the door), <code>fusion.py</code> (mix camera + weight + sensor signals, lost-sales ₹).
core/	Shared basics: event format, message bus (MQTT-ready), clock, geometry, config.
store/	SQLite database and reports.
api/	Web server (FastAPI) and the dashboard page (<code>static/</code>).
alerts/ , health/	Alert rules (no spam, escalation) and system health (camera frozen, temperature).
eval/	Accuracy tests, old-vs-new comparison, speed tests. Writes <code>RESULTS.md</code> .
firmware/stm32/	STM32 code (to be written next).

Try it: `cd C:\SIH\storemind` then `.\.venv\Scripts\python -m storemind.run --config configs\demo.yaml --backend scripted --realtime --api` and open `http://localhost:8000` .

4. Results so far (told honestly)

We label every number by where it came from. This matters a lot for the judges.

A: public benchmark

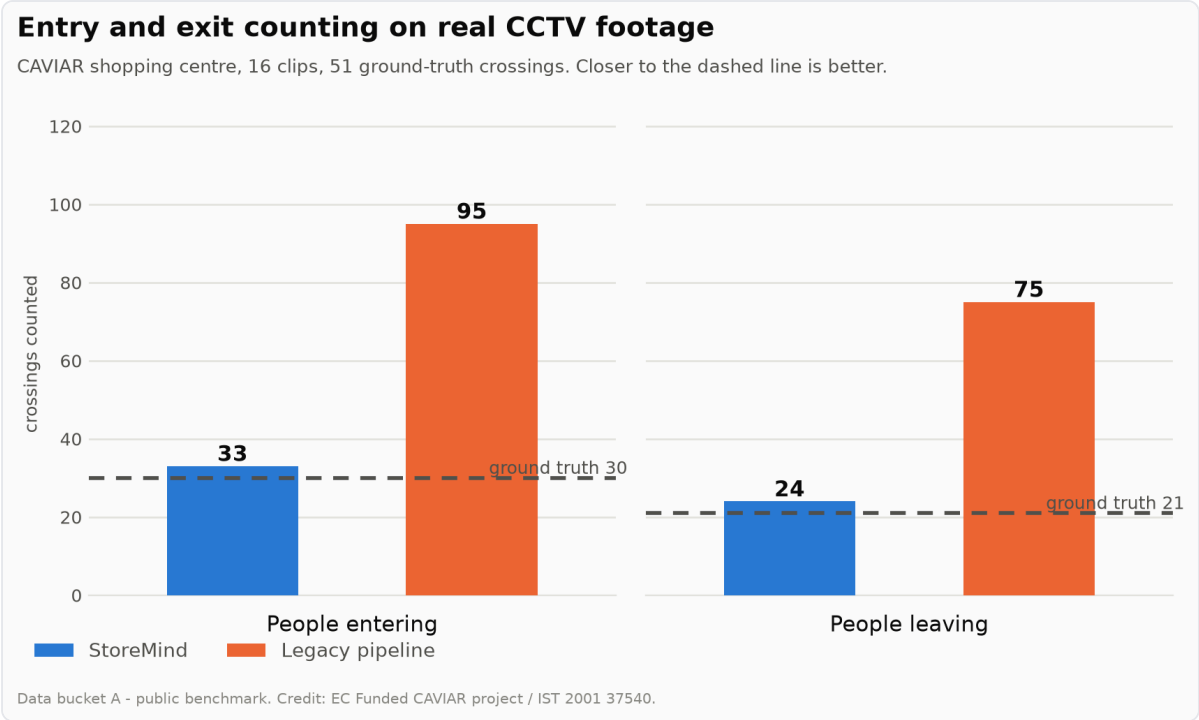
Real video with a published answer key (CAVIAR). Real accuracy.

C: simulation

Made-up videos. Proves the logic is right. **Not** real accuracy.

S: speed only

Real video, no answer key. Speed only.

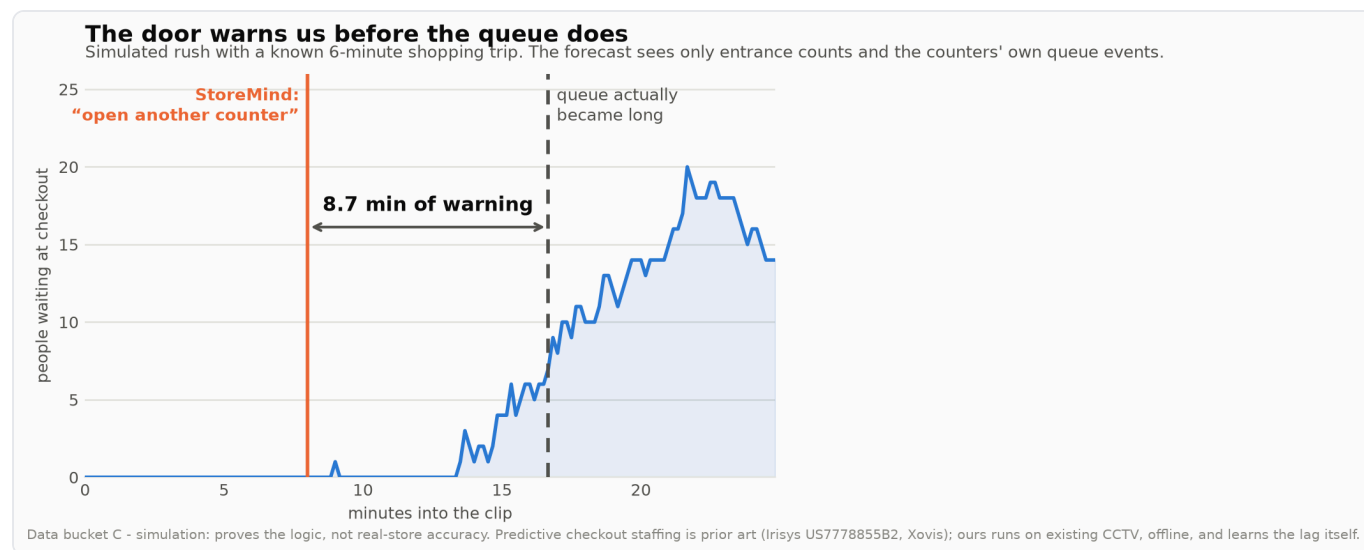


Real mall CCTV (CAVIAR, 16 clips). Blue = new StoreMind, orange = old code. Closer to the dashed line = better.

What we measured	Result	Type
People entering counted correctly	90.0% (33 counted, truth 30)	A
People leaving counted correctly	85.7% (24 counted, truth 21), below our 90% target	A
Old code on the same videos	95 entries vs truth 30, created 1,398 IDs for 111 people	A
People-in-frame error	0.65 people on average	A
Queue length / wait-time logic	0.00 people error / 0.39 s error	C
Shelf slot logic	95.8% correct, 0 false alerts when a person blocked the shelf	C
Queue forecast	Warned 8.7 minutes before the queue got long, 0 false alarms	C
Speed on laptop (YOLO11n, 320 px)	about 27 ms per frame (≈36 frames/second)	S

Honest notes: (1) We still count a few extra people near the line; fix planned. (2) CAVIAR is an easy, quiet mall; a busy kirana is harder. (3) Queue and shelf have **no real-world numbers yet**. We must record our own canteen and shelf videos. (4) Laptop speed changes a lot with heat; we need Pi and Qualcomm numbers.

The queue forecast idea in one picture



People who enter the shop now reach the billing counter a few minutes later. So the entrance camera can warn us early. (Simulation.) Note: big companies (Irisys, Xovis) already do this with costly sensors. Our difference: it runs on the CCTV a small shop already has, offline, on a cheap box, and learns the delay by itself.

What the system sees (privacy view)



Green boxes = detected people (blurred), red lines = counting line with buffer zone. Look at the man next to box #0: he was not detected, so he is not blurred. This is why our real privacy promise is "we never save frames", not "we blur everyone".

5. Hardware: what each part does

Part	Job	Status
Raspberry Pi 5	Main brain now: runs cameras, AI, database, dashboard.	have it
Qualcomm board (RUBIK Pi 3 or Radxa Dragon Q6A, chip QCS6490)	Final brain. Its AI chip (NPU) was measured 8.7× faster and 11× more energy efficient than a Pi 5 in a published test. It uses the same Pi camera and the same 40-pin header , so our wiring moves over unchanged.	to decide
STM32 Blue Pill + FreeRTOS	Real-time nerves: reads sensors, drives tower light and buzzer, sends small messages to the Pi over UART (serial). It does NOT run camera AI (too small: 20 KB RAM).	code next
HX711 + load cell	Weighs shelf trays or grain bins. Catches "back of shelf empty" that the camera cannot see.	worked before
ToF sensor (VL53L0X/L1X)	Sees a hand reaching into the shelf, so we know a real pickup happened.	planned
IR break-beam	Counts people at the door. Also our "answer key" to check camera counting.	planned
mmWave radar HLK-LD2450 (₹999)	Tracks up to 3 people at the billing counter with no camera . Works in the dark and when the camera is blocked.	new idea
MEMS microphone	Proposed job: hear the UPI soundbox saying "payment received". Each chime = one bill done, so we get sales count and billing time without a POS system . Never records audio. Needs an ESP32-S3 or a USB mic (Blue Pill has no I2S audio input).	to decide
Tower light + buzzer	Green / amber / red at the counter: "open another counter".	planned
Cameras	Shop CCTV, a Pi Camera Module 3, optional cheap ESP32-S3 shelf cameras.	see section 6
PCB	One board to hold the Blue Pill and all sensor connectors. Design in KiCad now (for the PPT), make it later.	to decide
Jetson Nano	Dropped as main device. Its software is end-of-life, and NVIDIA in a Qualcomm problem looks bad. Only an optional "second shop" demo.	dropped

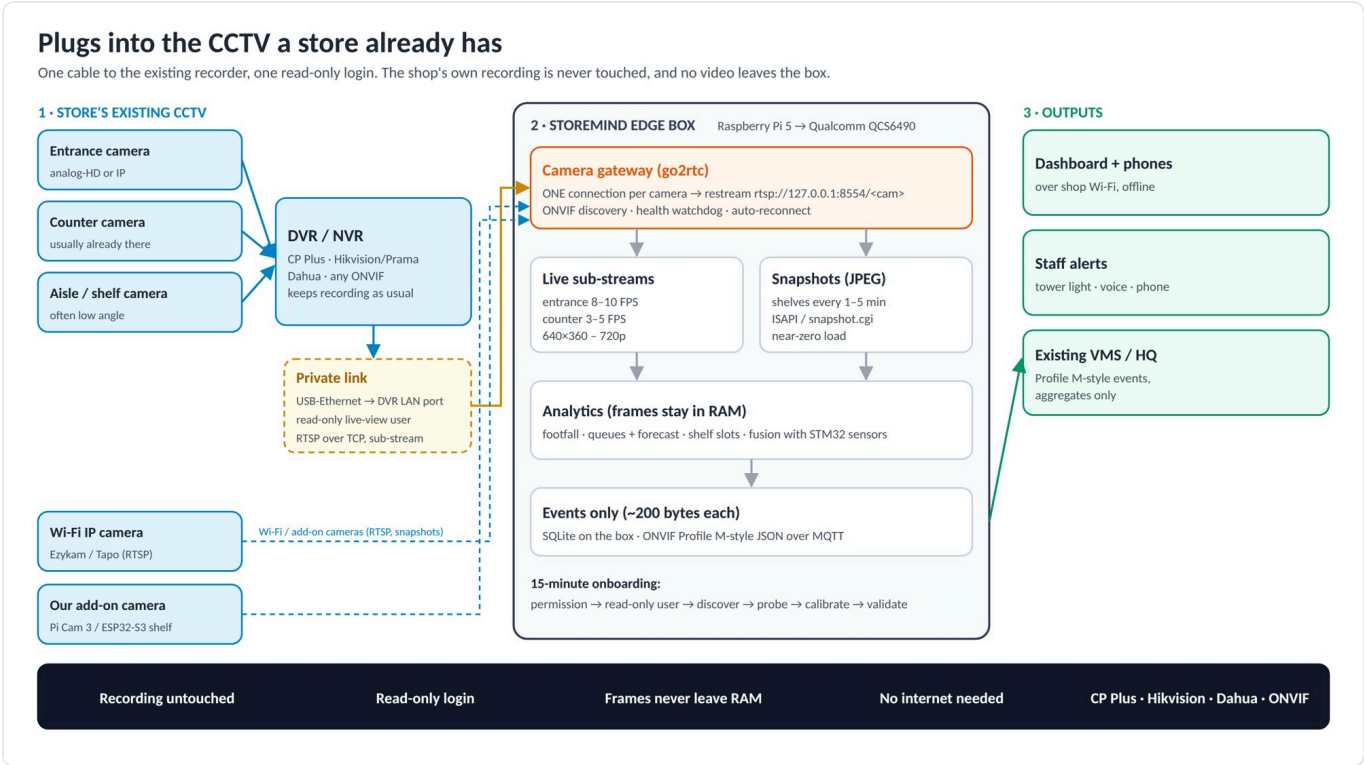
STM32 wiring plan (fits the Blue Pill)

STM32 pin group	Connects to	Note
USART1 (PA9/PA10)	Pi UART (pins 8/10)	115200 baud, 3.3 V, common ground
USART2 (PA2/PA3)	LD2450 radar	256000 baud; ignore targets outside the counter area
USART3 (PB10/PB11)	RS-485 module (optional)	for long cables to far shelves
I2C1 (PB6/PB7)	ToF sensors	use XSHUT pins to give each sensor its own address
GPIO pins	HX711, IR beam, tower light (via MOSFET), buzzer	HX711 read when its "data ready" pin goes low (interrupt)

Message format to the Pi (easy to read in a serial monitor): `$W,1,3876.2*5A` = weight sensor 1 reads 3876.2 g. Others: `$T` distance, `$B` door beam, `$Q` radar, `$R` restocked button, `$H` heartbeat.

6. Connecting to a shop's CCTV cameras

Simple idea: the shop's DVR/NVR (the CCTV recorder) already sends out every camera's video. We plug our box into the recorder with one cable, log in with a **read-only** account, and read a **low-quality copy** (sub-stream) of each camera. Their recording keeps working as usual. Like one more TV in the room.



How we connect (6 ways)

#	Way	When we use it
1	RTSP stream from the DVR/NVR (default)	Any recorder on a network. One link per camera.
2	ONVIF auto-discovery	Finds cameras and their stream links automatically.
3	Photo (snapshot) every 1-5 minutes	For shelves. Shelves change slowly, so no video needed.
4	Wi-Fi camera directly	Small shops with CP Plus Ezykam, Tapo, etc.
5	HDMI capture card	Very old DVR with no network.
6	Our own camera	Pi Camera Module 3 or ESP32-S3 where no CCTV looks.

Example stream links (check on the real device)

Brand	Low-quality live stream (RTSP)
Hikvision / Prama	<code>rtsp://user:pass@IP:554/Streaming/Channels/102</code> (camera 2 → 202)
CP Plus / Dahua	<code>rtsp://user:pass@IP:554/cam/realmonitor?channel=1&subtype=1</code>
Tapo Wi-Fi camera	<code>rtsp://user:pass@IP:554/stream2</code>

15-minute setup in a shop

1. **Permission letter** from the owner, and a privacy notice sign at the door.
2. Make a **read-only** login on the recorder.
3. **Find** the cameras (ONVIF scan or network scan).
4. **Test** each stream (`camera_check.py` shows real FPS and delay).
5. Add cameras to the **camera gateway (go2rtc)**: it keeps only one connection per camera, so the recorder is never overloaded.
6. **Draw** the counting line, queue lane and shelf slots on a snapshot (calibration tool).
7. **Check**: count 20–30 people by hand and compare with the dashboard.

Useful facts about Indian CCTV

- **CP Plus** is now the market leader (about 45% share in early 2026), so we support it first.
- A new government security certification rule (STQC) changed the market. Many shops still have older Hikvision/Dahua recorders, so we support those too.
- Kiranas usually already have cameras on the **cash counter and the door**, which are exactly our queue and footfall views.

Safety rules: read-only login · never change their recording · no internet port opening · private cable from our box to the recorder · nothing saved. **For the SIH demo:** one cheap Wi-Fi camera with RTSP + the Pi camera live, and recorded clips played as fake CCTV as a backup.

7. What makes us different (novelty)

Idea	Why it matters
Works on the CCTV shops already have	No new cameras needed. Cheap box (target ₹15–25k kit). Offline.
Queue forecast from the door	Warns before the queue forms. <i>Honest</i> : big companies (Irisys, Xovis) do this with costly sensors. We bring it to small shops, and the system learns the door-to-counter delay by itself.
Shelf check with one photo, no training	Take one "full shelf" photo per slot. A new product needs no retraining. Ignores shoppers blocking the view.
Camera + weight + radar together	Each sensor covers another's blind spot: weight sees the back of the shelf; radar works in the dark; the camera sees the whole store.
UPI soundbox listener (proposed)	94% of small merchants use UPI. Hearing "payment received" gives sales count and billing time with no POS system. Very India-specific.
Everything in rupees	"Shopper stood at an empty shelf" → estimated lost sale in ₹. Owners understand money.
Built for Qualcomm	Models tested on Qualcomm AI Hub, run on the Qualcomm NPU, speed and energy measured.
Privacy by design	No faces, no identity, no saved video, "video stored: 0 bytes" shown live on the dashboard.

Words to avoid on slides: "first in the world", "behaviour analysis" (sounds like spying), "Jetson". **Words to use:** "anonymous footfall", "works on existing CCTV", "offline", "Qualcomm NPU", "measured".

8. Problems we expect, and how we will fix them

Problem	Our fix
People standing near the door line get counted twice	Two lines instead of one (must cross both in order), a direction check, and a minimum walking distance.
The tracker loses people in a crowd	Test better trackers (OC-SORT, BoT-SORT) against ByteTrack and pick the best on real video.
Staff get counted as customers	Ignore the zone behind the counter. Staff wear a printed pattern badge (like Xovis does). We never identify who they are.
People walking past the queue get counted as waiting	Only count someone who is slow and stays at least a few seconds.
A family of 3 counts as 3 customers	Group people who stand together and leave together into one "party".
Bent or long queues	Draw the queue lane as a path, not a box, and watch the tail.
People hide each other at peak time	Mount the camera high, add the radar at the counter, and cross-check wait time with Little's law (average wait = queue length ÷ arrival rate).
Camera cannot see when billing ends	UPI soundbox listener, POS file, or "customer left the billing spot".
Shelf lighting changes (day/night)	Compare edges, not raw colours; keep day and night reference photos.
Far shelves are too small in CCTV	Optional cheap ESP32-S3 shelf cameras that send a photo every few minutes.
Recorder allows only a few connections	Camera gateway (go2rtc): one connection per camera, shared by everything.
Camera freezes or is moved	Auto-reconnect, "frozen camera" alarm, "camera moved" detection.
Internet or power goes off	Everything works offline; database is safe on power loss; UPS; auto-restart.
Laptop speed numbers change a lot	Measure on the Pi and on Qualcomm, all in one sitting, and report energy per frame too.

9. What comes next, and how you can help

Build order (after the team agrees)

#	Work	Done when
1	Better door counting + tracker test	Exit accuracy $\geq 90\%$ on videos we did not tune on
2	Camera gateway, fake-CCTV test setup, "how many cameras per box" test	4 streams run for 1 hour without lag
3	Smarter queue (passers-by, groups, bent queues, Little's law, staff zone)	Queue error ≤ 1 person on our canteen video
4	STM32 link: simulator + FreeRTOS firmware + radar + tower light	Real board drives the dashboard for 1 hour
5	UPI chime listener (if approved)	$\geq 90\%$ chimes detected, no false alarms in shop noise
6	Shelf v2 (day/night), optional ESP32 shelf camera	Empty-slot score ≥ 0.85 on our demo shelf
7	Power/energy panel, auto-start, 24-hour test, "pull the plug" tests	Survives all tests
8	Qualcomm AI Hub + Qualcomm board	Speed/energy table: Pi 5 vs Qualcomm
9	"Ask your store" (local AI answers owner questions), reorder list, shelf model training	20 test questions, 0 made-up numbers

5 decisions we need from the team

1. **Qualcomm board:** buy or borrow a RUBIK Pi 3 or Radxa Dragon Q6A?
2. **MEMS sensor:** use it as the UPI soundbox listener? (needs an ESP32-S3 or a USB mic)
3. **ESP32-S3 shelf cameras:** in or out?
4. **PCB:** design in KiCad now (for the PPT), make it later?
5. **Test site:** which canteen or shop, and who asks for permission?

Jobs anyone can pick up now

Record videos

Canteen queue and a snack shelf, 5–10 minutes each, phone fixed high. Count by hand (templates in [videos/GROUND_TRUTH_TEMPLATES/](#)).

Hardware

Add the old HX711 code to GitHub. Order the LD2450 radar (₹999) and a USB-C power meter. Try the Blue Pill in Wokwi (online simulator).

Qualcomm

Sign up for Qualcomm AI Hub (free) and share the API token with Vinith privately.

Field visits

Talk to 3–5 shop owners: do you have CCTV? biggest problem? would you pay ₹15–20k?

How to send suggestions

Open GitHub Desktop → make your own branch (`yourname/idea`) → change or add a file (for example `research/suggestions_yourname.md`) → Push → "Create Pull Request". Vinith reviews and merges. Quick ideas are fine in the group chat too.

10. Simple glossary

Word	Meaning in simple words
Edge / edge AI	AI that runs inside the shop on a small box, not on the internet.
NPU	A special chip part made only for AI. Much faster and cooler than the normal CPU.
INT8 / quantization	Making the AI model smaller (8-bit numbers) so it runs fast on an NPU.
YOLO / detector	The AI model that draws boxes around people in each frame.
Tracker (ByteTrack, OC-SORT, BoT-SORT)	Links the same person across frames, giving each a temporary number.
FPS	Frames per second: how many pictures per second we process.
RTSP	The standard way CCTV cameras send live video over a network.
DVR / NVR	The CCTV recorder box. DVR for older analog cameras, NVR for IP cameras.
Sub-stream	The low-quality copy of a camera's video. Enough for AI, light on the network.
ONVIF	A common "language" many cameras speak, used to find them and get their stream links.
go2rtc	A small program that connects once to each camera and shares the video with everything else.
MQTT	A light messaging system: parts of our system send each other small messages.
SQLite	A small database stored as one file on the box.
FastAPI / dashboard	The web server and the web page that show live numbers.
STM32 / FreeRTOS	A small microcontroller and the mini operating system that lets it do many sensor jobs at once.
UART	Simple 2-wire serial connection between the STM32 and the Pi.
HX711	Chip that reads a load cell (weight sensor).
ToF	Time-of-Flight: a laser distance sensor.
mmWave radar	Radio sensor that detects moving people without a camera.
Erlang-C	A queueing formula (from call centres) that tells how many counters are needed for a target waiting time.
Little's law	Average wait = average queue length ÷ how many people arrive per minute.
Accuracy, F1, MAE	Ways to score results. F1 near 1.0 is good; MAE is the average error (smaller is better).
Overfitting	When a model memorises its training images and fails on new ones.
CAVIAR	A public set of real mall CCTV videos with a published answer key.
Qualcomm AI Hub	Free Qualcomm website that runs our model on real Qualcomm chips and reports speed.
DPDP Act	India's data privacy law. Why we never store faces or video.

Sources and full details: [research/](#) folder (files 00–24). Key reports: 20 (Qualcomm + market), 22 (Gemini review + prior art), 23 (pipeline problems), 24 (CCTV).